

Tenzorske metode za Recommender Systems

Nikola Mamić, Petar Matejčić i Jakov Tušek

Veljača 2026.

1 Uvod

U ovom radu bavimo se algoritmima koji korisniku preporučuju oznake za određenu stavku na temelju njegovih prethodnih oznaka za druge stavke i oznaka drugih korisnika. Primjerice, može se raditi o dodjeljivanju žanrova filmovima ili pjesmama. Pritom su ponude personalizirane tj. ovise o korisnikovim preferencijama, a ne globalne. Međutim, pristup u radu je općenit pa se isti algoritmi mogu koristiti za široku klasu problema rangiranja kao što su preporuke stavki, pretraživanje i sl. Koristimo Bayesian Personalized Ranking model u kombinaciji s HOSVD, CD te PITF (Pairwise Interaction Tensor Factorization) dekompozicijama tenzora, a treniranje se provodi verzijom SGD algoritma. Na kraju, testiramo prezentirane algoritme na konkretnim primjerima filmova, glazbe, stranica za upoznavanje i stranica za social bookmarking.

2 Formalizacija problema i oznake

Neka je U skup svih korisnika, I skup svih stavki i T skup svih oznaka. Neka je $S \subseteq U \times I \times T$ pri čemu je $(u, i, t) \in S$ ako je korisnik u označio stavku i oznakom t . Naš cilj je, na temelju S , za svaki par $(u, i) \in U \times I$ procijeniti latentni totalni uređaj $>_{u,i}$ na T koji interpretiramo kao redoslijed preferencije oznaka iz T za stavku i kod korisnika u . Tada možemo korisniku preporučiti top n oznaka prema tom uređaju.

Navedeno ćemo ostvariti modeliranjem funkcije $y: U \times I \times T \times T \rightarrow \mathbb{R}$ koja će za četvorku (u, i, t_A, t_B) govoriti koliko je "vjerojatno" da je $t_A >_{u,i} t_B$. Ona će biti parametarski modelirana preko odgovarajuće tenzorske dekompozicije.

Potreban nam je još skup $D_S := \{(u, i, t_A, t_B) : (u, i, t_A) \in S \wedge (u, i, t_B) \notin S\}$. Ovo je zapravo skup za treniranje jer on nam govori informacije koje imamo o našem uređaju: Ako je $(u, i, t_A, t_B) \in D_S$ to znači da je korisnik u za stavku i preferirao oznaku t_A nad oznakom t_B tj. odabrao je t_A , a nije odabrao t_B . Ovakav pristup se pokazuje puno boljim od jednostavno proglašavanja svih odabranih stavki jedinicama, a neodabranih nulama jer se ne ignorira činjenica da nisu sve odabrane oznake jednako poželjne i, još važnije, nisu sve neodabrane jednako nepoželjne.

3 Bayesian Personalized Ranking (BPR)

Kao što je ranije komentirano, funkcija y bit će zadana parametarski preko parametara Θ . Sada želimo izvesti izraz za aposteriornu distribuciju parametra koju ćemo onda u konkretnim primjerima maksimizirati i tako ih procijeniti. Iz Bayesovog teorema imamo:

$$p(\Theta | >_{u,i}) \propto p(>_{u,i} | \Theta) p(\Theta)$$

Što se tiče $p(\Theta)$ pretpostavit ćemo da je apriorna distribucija normalna tj. $\Theta \sim N(0, \sigma_\Theta^2 I)$. S $p(>_{u,i} | \Theta)$ naizgled ne možemo napraviti puno jer nam u ovom trenutku nije uopće poznato kako izgleda model ovisno o parametrima tj. razvijamo makismalno općenit pristup. Ono što možemo je modelirati to kao:

$$p(>_{u,i} | \Theta) = \prod_{(u,i,t_A,t_B) \in D_S} p(t_A >_{u,i} t_B | \Theta)$$

Time koristimo informaciju da naša relacija zadovoljava one odnose koje imamo u trening setu i kada kasnije maksimiziramo aposteriornu distribuciju zapravo odabiremo one parametre Θ za koju su ti odnosi maksimalno vjerodostojni.

Ostaje nam modelirati $p(t_A >_{u,i} t_B | \Theta)$, no kao što smo već rekli naša funkcija y bi trebala, jednom kad odredimo parametre, govoriti koliko je "vjerojatno" da je $t_A >_{u,i} t_B$ pa to možemo modelirat kao:

$$p(t_A >_{u,i} t_B | \Theta) = \sigma(y_{(u,i,t_A,t_B)}(\Theta))$$

gdje je npr. $\sigma(x) = \frac{1}{1+e^{-x}}$ (želimo strogo rastuću transformaciju y u nešto što će zaista biti distribucija, a što ćemo kasnije moći derivirati). Ovdje $y_{(u,i,t_A,t_B)}(\Theta)$ zapravo označava vrijednost funkcije y u (u,i,t_A,t_B) kada je parametar Θ (funkcija će biti određena s Θ) a takva je notacija jer će nas kasnije zanimati ponašanje toga ovisno o Θ .

Konačno imamo:

$$\ln(p(\Theta | >_{u,i})) = \sum_{(u,i,t_A,t_B) \in D_S} \ln \sigma(\hat{y}_{u,i,t_A,t_B}(\Theta)) - \lambda_\Theta \|\Theta\|_F^2$$

I to želimo maksimizirati po Θ i tako dobiti parametre.

3.1 Metoda stohastičkog najbržeg silaska

Općenito, traženje maksimuma navedene funkcije cilja je vremenski zahtjevan zadatak zbog veličine skupa D_S koji vrlo brzo postane jako velik (reda veličine

$O(|S||T|)$.

Iz tog razloga, za maksimiziranje funkcije cilja koristimo metodu stohastičkog najbržeg silaska. Glavna ideja te metode je da gradijent ∇f funkcije oblika $f = \sum f_i$ zamijenimo gradijentom ∇f_i za neki slučajno odabrani i te radimo korak u smjeru $-\nabla f_i$. Ova metoda nije metoda silaska, ali uz određene uvjete na način odabira indeksa i možemo postići silazak u očekivanju.

Jedan način da osiguramo silazak u očekivanju je da indekse i u svakom koraku biramo nezavisno u odnosu na prethodne i uniformno na skupu svih indeksa. Konkretno, naš algoritam u svakom koraku mora uniformno birati $(u, i, t_A, t_B) \in D_S$.

Dakle, algoritam za maksimiziranje dane funkcije cilja je:

```

procedura LEARNBPR( $D_S$ )
   $\Theta \sim N(0, \sigma_\Theta I)$ 
  ponavlja
    izvuci  $(u, i, t_A, t_B)$  uniformno iz  $D_S$ 
     $\Theta \leftarrow \Theta + \alpha \frac{\partial}{\partial \Theta} (\ln \sigma(\hat{y}_{u,i,t_A,t_B}) - \lambda_\Theta \|\Theta\|_F^2)$ 
  dok konvergencija
  vрати  $\hat{\Theta}$ 
kraj procedura

```

3.2 Faktorizacije

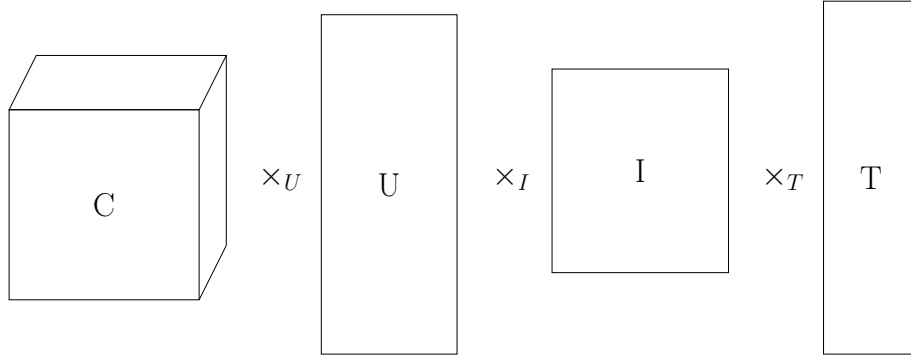
Gore navedena funkcija cilja i pripadni algoritam *LearnBPR* mogu se koristiti za razne modele, no mi ćemo se baviti modelima baziranim na tenzorskim faktorizacijama. Odnosno, želimo odrediti funkciju $Y : U \times I \times T \rightarrow \mathbb{R}$ koja uređenoj trojci (u, i, t) pridružuje koeficijent relevantnosti, u smislu koliko je oznaka t relevantna korisniku u za stavku i . Prirodno onda Y prikazujemo kao tenzor, gdje je $y_{u,i,t}$ koeficijent relevantnosti pripadne uređene trojke, a za *LearnBPR* definiramo:

$$y_{u,i,t_A,t_B} := y_{u,i,t_A} - y_{u,i,t_B}$$

Tuckerova dekompozicija

Iz Tuckerove dekompozicije dobijemo izraz za $y_{u,i,t}$:

$$\hat{y}_{u,i,t}^{TD} = \sum_{u'} \sum_{i'} \sum_{t'} \hat{c}_{u',i',t'} \hat{u}_{u,u'} \hat{i}_{i,i'} \hat{t}_{t,t'},$$



Slika 1: Tuckerova dekompozicija

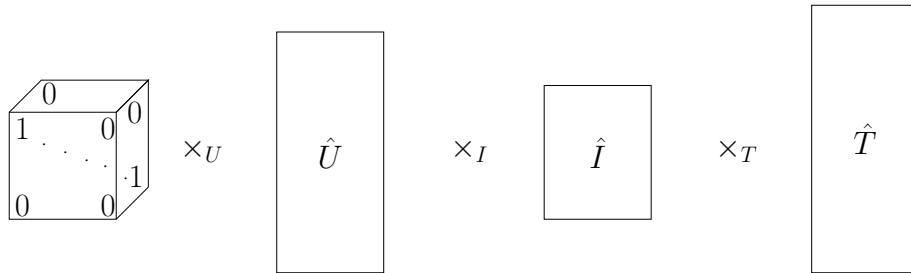
gdje su parametri modela:

$$\begin{aligned}\hat{C} &\in \mathbb{R}^{k_u \times k_i \times k_t}, & \hat{U} &\in \mathbb{R}^{|U| \times k_u} \\ \hat{I} &\in \mathbb{R}^{|I| \times k_i}, & \hat{T} &\in \mathbb{R}^{|T| \times k_t}\end{aligned}$$

Tada su gradijenti $\frac{\partial y_{u,i,t}^{TD}}{\partial \Theta}$ za *LearnBPR* dani s:

$$\begin{aligned}\frac{\partial \hat{y}_{u,i,t}^{TD}}{\partial \hat{c}_{\tilde{u},\tilde{i},\tilde{t}}} &= \hat{u}_{u,\tilde{u}} \hat{i}_{i,\tilde{i}} \hat{t}_{t,\tilde{t}}, & \frac{\partial \hat{y}_{u,i,t}^{TD}}{\partial \hat{u}_{u,\tilde{u}}} &= \sum_{\tilde{i}} \sum_{\tilde{t}} \hat{c}_{\tilde{u},\tilde{i},\tilde{t}} \hat{i}_{i,\tilde{i}} \hat{t}_{t,\tilde{t}} \\ \frac{\partial \hat{y}_{u,i,t}^{TD}}{\partial \hat{i}_{i,\tilde{i}}} &= \sum_{\tilde{u}} \sum_{\tilde{t}} \hat{c}_{\tilde{u},\tilde{i},\tilde{t}} \hat{u}_{u,\tilde{u}} \hat{t}_{t,\tilde{t}}, & \frac{\partial \hat{y}_{u,i,t}^{TD}}{\partial \hat{t}_{t,\tilde{t}}} &= \sum_{\tilde{u}} \sum_{\tilde{i}} \hat{c}_{\tilde{u},\tilde{i},\tilde{t}} \hat{u}_{u,\tilde{u}} \hat{i}_{i,\tilde{i}}\end{aligned}$$

CP dekompozicija



Slika 2: CP dekompozicija

CP dekompozicija nam daje izraz za $y_{u,i,t}$:

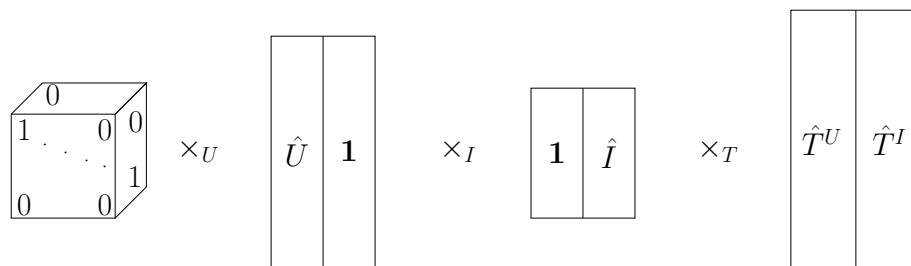
$$\hat{y}_{u,i,t}^{CP} = \sum_f^k \hat{u}_{u,f} \hat{i}_{i,f} \hat{t}_{t,f},$$

pri čemu je $k := \min\{k_u, k_i, k_t\}$.

Gradijenti $\frac{\partial y_{u,i,t}^{CP}}{\partial \Theta}$ za *LearnBPR* su:

$$\begin{aligned}\frac{\partial \hat{y}_{u,i,t}^{\text{CP}}}{\partial \hat{u}_{u,f}} &= \hat{i}_{i,f} \cdot \hat{t}_{t,f} \\ \frac{\partial \hat{y}_{u,i,t}^{\text{CP}}}{\partial \hat{i}_{i,f}} &= \hat{u}_{u,f} \cdot \hat{t}_{t,f} \\ \frac{\partial \hat{y}_{u,i,t}^{\text{CP}}}{\partial \hat{t}_{t,f}} &= \hat{u}_{u,f} \cdot \hat{i}_{i,f}\end{aligned}$$

PITF (Pairwise Interaction Tensor Factorization)



Slika 3: PITF

PITF tenzorska faktorizacija modelira dvostrane interakcije između korisnika, stavki i oznaka, na način da izraz za $y_{u,i,t}$ možemo rastaviti na tri sume:

$$\hat{y}_{u,i,t} = \sum_f \hat{u}_{u,f} \hat{t}_{t,f}^U + \sum_f \hat{i}_{i,f} \hat{t}_{t,f}^I + \sum_f \hat{u}_{u,f} \hat{i}_{i,f}$$

Uočimo, za fiksne (u, i) treća suma nam ne utječe na optimizaciju funkcije cilja ni na konačni poredak oznaka jer je konstantna za sve oznake t . Stoga će nam konačni model koeficijenta relevantnosti biti:

$$\hat{y}_{u,i,t} = \sum_f \hat{u}_{u,f} \hat{t}_{t,f}^U + \sum_f \hat{i}_{i,f} \hat{t}_{t,f}^I,$$

gdje su parametri modela:

$$\begin{aligned}\hat{U} &\in \mathbb{R}^{|U| \times k}, & \hat{I} &\in \mathbb{R}^{|I| \times k} \\ \hat{T}^U &\in \mathbb{R}^{|T| \times k}, & \hat{T}^I &\in \mathbb{R}^{|T| \times k}\end{aligned}$$

Gradijenti $\frac{\partial y_{u,i,t}^{PITF}}{\partial \Theta}$ za *LearnBPR* su:

$$\begin{aligned}\frac{\partial \hat{y}_{u,i,t}}{\partial \hat{u}_{u,f}} &= \hat{t}_{t,f}^U, & \frac{\partial \hat{y}_{u,i,t}}{\partial \hat{i}_{i,f}} &= \hat{t}_{t,f}^I \\ \frac{\partial \hat{y}_{u,i,t}}{\partial \hat{t}_{t,f}^U} &= \hat{u}_{u,f}, & \frac{\partial \hat{y}_{u,i,t}}{\partial \hat{t}_{t,f}^I} &= \hat{i}_{u,f}\end{aligned}$$

4 Primjeri

Za kraj, primijenili smo navedene algoritme na nekoliko skupova podataka.

Za usporedbu algoritama i istraživanje utjecaja parametara koristili smo *f1*-mjeru. Kako nam je cilj odrediti top N najrelevantnijih oznaka, za svaki par (u, i) iz testnog skupa određujemo N najboljih oznaka prema našem modelu te gledamo koliko njih se zapravo pojavilo u podacima.

$$\text{Prec}(S_{\text{test}}, N) := \text{avg}_{(u,i) \in P_{S_{\text{test}}}} \frac{|\text{Top}(u, i, N) \cap \{t \mid (u, i, t) \in S_{\text{test}}\}|}{N}$$

$$\text{Rec}(S_{\text{test}}, N) := \text{avg}_{(u,i) \in P_{S_{\text{test}}}} \frac{|\text{Top}(u, i, N) \cap \{t \mid (u, i, t) \in S_{\text{test}}\}|}{|\{t \mid (u, i, t) \in S_{\text{test}}\}|}$$

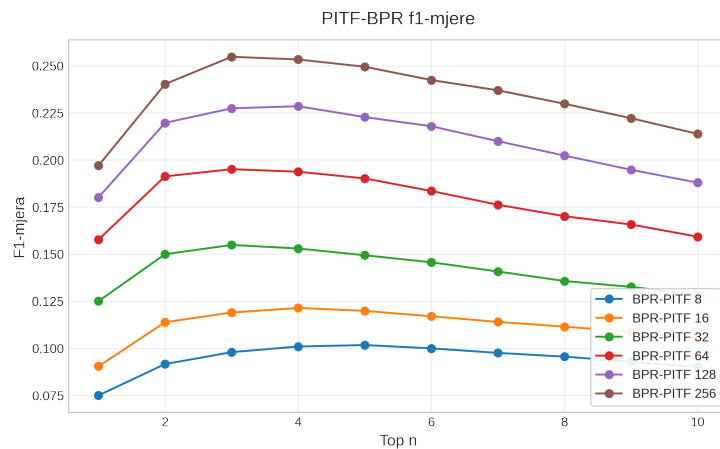
$$\text{F1}(S_{\text{test}}, N) := \frac{2 \text{Prec}(S_{\text{test}}, N) \text{Rec}(S_{\text{test}}, N)}{\text{Prec}(S_{\text{test}}, N) + \text{Rec}(S_{\text{test}}, N)}$$

Dijelimo skupove podataka na skup za treniranje i testiranje tako da za svakog korisnika uklonimo jednu objavu/interakciju (post) i stavimo ga u skup za testiranje. Na kraju provjerimo je li model istreniran na preostalim podacima točno predvidio objave/interakcije iz skupa za testiranje.

4.1 Movielens

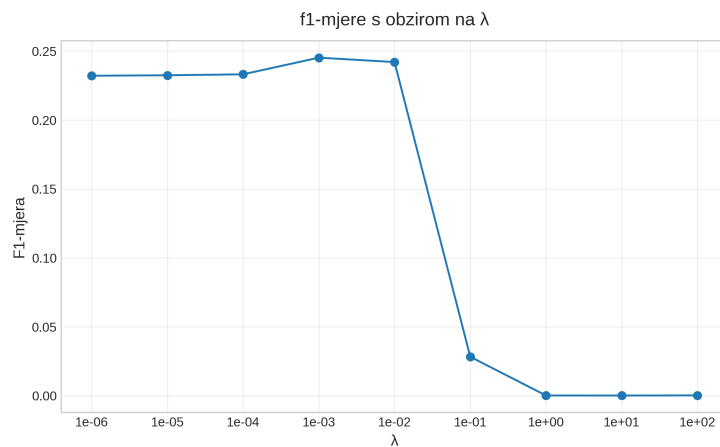
Testirali smo prethodno definirane algoritme na podacima o korisničkim oznakama pridruženih filmovima. Podaci su prikupljeni sa stranice GroupLens, te sadrže sveukupno 87,500 filmova, 200,000 korisnika i 2,000,000 oznaka. No, zbog činjenice da korisnici sami generiraju oznake, većina tih oznaka se pojavljuje tek nekoliko puta. Zbog toga smo podatke smanjili samo na one korisnike, filmove i oznake koji se pojavljuju barem 7 puta u datasetu, pa nam je konačni broj podataka: 4,304 korisnika, 7,475 filmova i 19,745 oznaka s 4,012,200 veza.

Za početak smo istražili utjecaj parametra k , tj. dimenzije matrica U , I , T^U i T^I , na kvalitetu predviđenih parametara. Na slici 4 vidimo da kvaliteta predviđanja raste s dimenzijom matrica u modelu, ali naravno raste i broj potrebnih operacija za izračun. Također, vidimo da svi modeli najbolje predviđaju oko top 3 oznake.



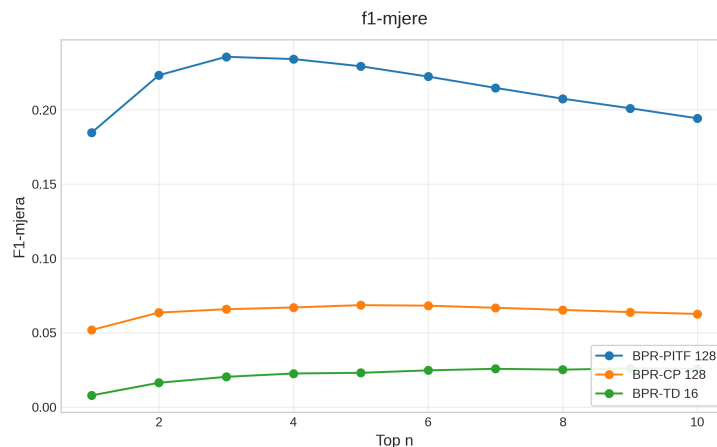
Slika 4: Utjecaj parametra k na kvalitetu modela

Zatim smo za model *BPR-PITF 128* proučili utjecaj parametra regularizacije λ na kvalitetu predviđanja. Na slici 5 vidimo da se model ponaša stabilno za λ blizu 0, tj. imamo jednako dobre aproksimacije.



Slika 5: Utjecaj parametra λ na kvalitetu modela

Konačno, usporedili smo *BPR-PITF 128*, *BPR-CP 128* i *BPR-TD 16*.



Slika 6: Usporedba modela

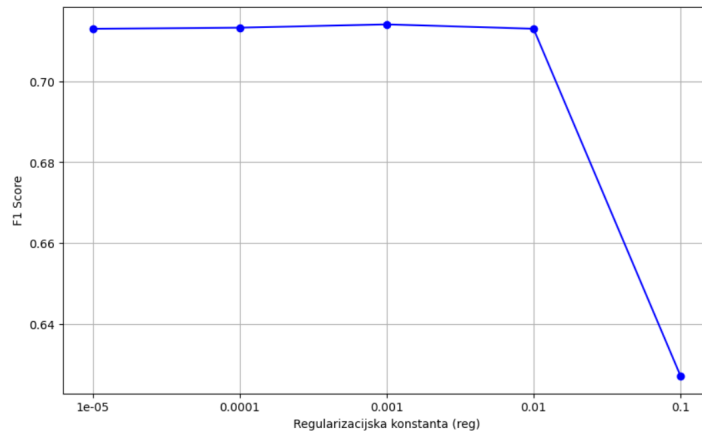
4.2 Libimseti

U ovom primjeru primjenjujemo opisane algoritme na podatke o ocjenama koje su korisnici dali drugim korisnicima na češkoj stranici za upoznavanje libimseti.cz. Ocjene su 1-10 i na raspolaganju imamo preko 17 milijuna ocjena te za svaku znamo koji korisnik ju je dodijelio kojem korisniku. Izbačeni su svi korisnici koji nisu dali bar 100 ocjena i dobili bar 10. Ocjene su grupirane u 3 razreda: 1-3, 4-6 i 7-10 jer smo smatrali da prva kategorija otprilike opisuje osobe koje se nikako ne sviđaju osobi, a zadnja koje joj se sviđaju. Napomenimo da su podaci bili balansirani tj. imali smo podjednako kategorija.

Cilj je opisane algoritme iskoristiti tako da predvidimo u koju bi kategoriju osoba u svrstala osobu i u svrhu preporučivanja osobi u osoba koje bi joj se mogle svidjeti. Dakle klasificirali smo osobu i u kontekstu osobe u u onu kategoriju koja je u kontekstu našeg predviđenog uređaja bila najveća.

4.2.1 PITF

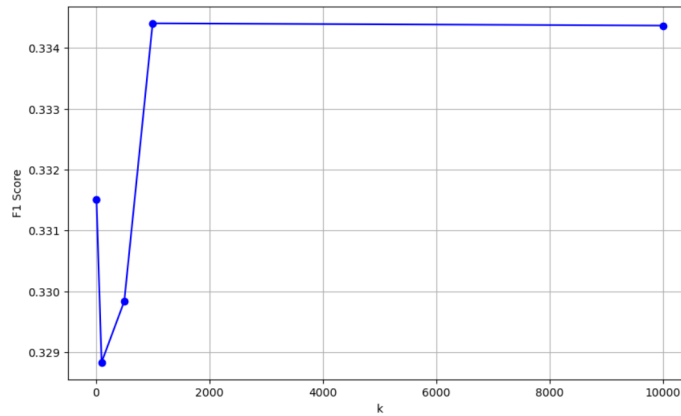
PITF je pokazao svoju snagu u ovom primjeru i producirao značajno bolje predikcije na test podacima od ostala dva algoritma s f1 scoreom uglavnom oko 0.7 ovisno o regularizacijskom parametru.



Slika 7: Utjecaj parametra λ na kvalitetu modela

4.2.2 Kanonska dekompozicija

Iako smo pokušali s različitim rangovima, kanonska dekompozicija je davala puno lošiji rezultat s f1 oko 0.33 ovisno o odabranom rangu aproksimacije.



Slika 8: Utjecaj parametra k na kvalitetu modela

4.2.3 Tuckerova dekompozicija

Ovaj model je davao slične rezultate kao i prethodni samo uz dosta dulje vrijeme treniranja.

4.3 Delicious (del.icio.us)

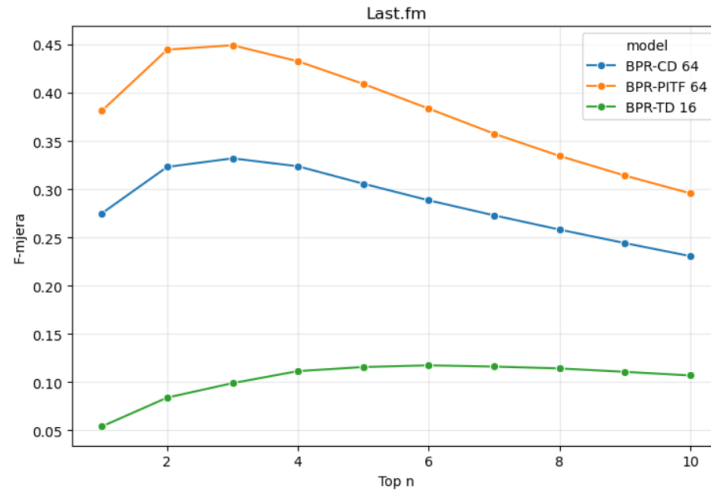
Primjenjujemo algoritme na podatke stranice za društveni *bookmarking* Delicious. U 2000-tim godinama stranica je bila popularan resurs za indeksiranje i pretraživanje stranica o raznim temama. Korisnici su mogli označavati indeksirane web-stranice proizvoljnim oznakama, starajući time tzv. *folksonomy*, "društvenu taksonomiju" interneta. Podaci su preuzeti sa stranice GroupLens i sastoje se od 437 tisuća veza, s gotovo 41 tisuća različitih oznaka za 69 tisuća web-dokumenata od 1867 korisnika. Problem kod ovakvih sustava u kojima korisnici sami generiraju oznake je ta što se, kao kod podataka za MovieLens, većina oznaka pojavljuje samo jednom ili dvaput (većina oznaka nisu popularne). Ovdje reduciramo skup na 3-*core* skup, što je najmanji skup gdje se svaki podatak u svakoj trojki pojavljuje barem 3 puta u skupu podataka. To reducira skup na 84 tisuće trojki od 1458 korisnika, 5074 web-dokumenata i 4233 oznaka.

Pokaže se izvođenjem PITF algoritma na 500 000 iteracija i $k=128$ da $F1@3$ metrika (na top 3 oznake) dostigne vrijednost 0.22, dok CP algoritam dostigne samo 0.04. Manja $F1$ metrika očekivana je za skupove podataka s mnogo oznaka i usporedivo manje podataka. Ipak, rezultati su značajno bolji od slučaja kada nasumično pogađamo oznake za dani par korisnika i web-dokumenta, gdje se pokaže da je na danom podacima Precision@5 iznosi svega 0.001. Dakle, već na malom skupu podataka s velikom razinom šuma (riječ je samo o 3-*core* skupu podataka) i malom broju iteracija imamo veliko poboljšanje u predviđanju.

4.4 Last.fm

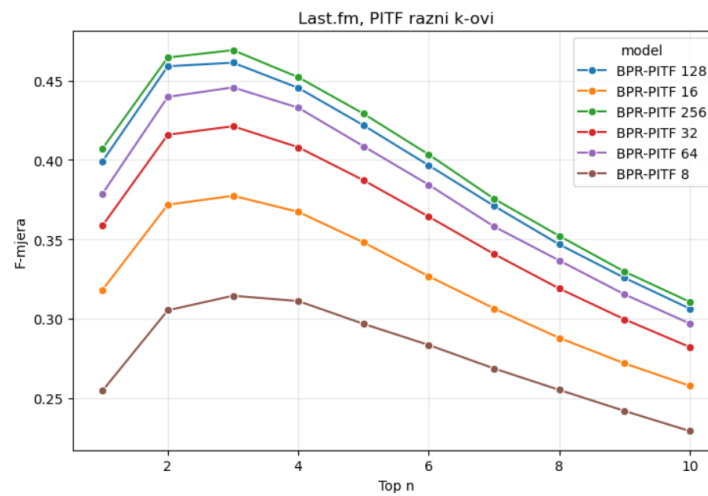
Last.fm je web-stranica koja korisnicima omogućava označavanje glazbenika/bendova, pjesama, albuma, radio stanica i sl. i recommender sustavom *Audioscrobbler*, kojim korisnicima na zahtjev preporuča novu glazbu/umjetnike/radio stanice ovisno o raznim metrikama. Jedan od podsustava omogućava korisnicima da stvore *playlist*-ove ovisno o oznakama. Tada sustav korisniku kreira popis pjesama na temelju pjesama koje je korisnik već označio tom oznakom. Podaci su preuzeti sa stranice GroupLens i sastoje se od 186479 zapisa (trojki), 9749 oznaka, 12 523 glazbenika/grupa, 1892 korisnika. Nakon stvaranja 7-*core* podskupa, preostaje 811 osoba, 2386 glazbenika/grupa, 1288 oznaka i sveukupno 111 149 zapisa.

Uspoređujemo performanse sva tri algoritma na istim podacima. Korišten je Adagrad optimizator za SGD, za PITF i CP korišteno je 700 000 iteracija, a za Tucker 200 000 (za približne performanse kao PITF i CP na 700 000 iteracija bilo bi potrebno više milijuna iteracija za Tuckera, što bi zahtijevalo puno vremena). $F1@N$ metrike istreniranih modela vidljivi su na grafu.



Slika 9: Usporedbe različitih modela na Last.fm podacima

Također ispitujeemo utjecaj odabira broja k , broja latentnih dimenzija koje otkrivamo, na performanse PITF modela. U radu [3] na sličnim podacima (Last.fm, ali 10-core i otprilike 4 puta više zapisa) autori su empirijsko pokazali da odabir većeg k nema znatnih poboljšanja na performanse, što se objašnjava time da za aproksimaciju koristimo previše latentnih dimenzija, pa "višak" postane blizu nuli - nema gotovo nikakvog utjecaja na rekonstrukciju. Testiran je PITF model na raznim vrijednostima k i rezultati su prikazani na grafu.



Slika 10: Usporedbe različitih modela na Last.fm podacima

4.5 Zaključak

Na primjerima smo pokazali kako je PITF model zadovoljavajućih preformansa, u smislu da konvergira ka boljem rješenju u manje iteracija od drugih predstavljenih modela. Potencijalna poboljšanja bila bi povećanje robusnosti na perturbacije/šum u podacima koristeći tzv. *adversarial* učenje, gdje se pri treniranju modela uključuju perturbacije parametara. Time bismo mogli koristiti još rjeđe i slabije povezane podatke, gdje se elementi pojavljuju manje puta.

Literatura

- [1] Evgeny Frolov and Ivan Oseledets. Tensor methods and recommender systems, 2018.
- [2] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. Bpr: Bayesian personalized ranking from implicit feedback, 2012.
- [3] Steffen Rendle and Lars Schmidt-Thieme. Pairwise interaction tensor factorization for personalized tag recommendation. In *Proceedings of the Third ACM International Conference on Web Search and Data Mining*, WSDM '10, page 81–90, New York, NY, USA, 2010. Association for Computing Machinery.